

Reinforcement Learning Using Transformers in TextWorld Environments

Sho Sato s1300221

Supervised by Prof. Naohito Nakasato

Abstract

Bridging the semantic gap between ambiguous natural-language instructions and formal executable commands remains a significant challenge in the development of autonomous agents. In this study, we evaluated the performance of Long Short-Term Memory (LSTM), Transformer, and Generative Pre-trained Transformer 2 (GPT-2) in generating executable commands within the TextWorld environment. We implemented a unified reinforcement learning framework based on Proximal Policy Optimization (PPO) and conducted experiments across three distinct scenarios, Exploration, Key-Door, and Common-Sense, allowing for a controlled comparison of memory, planning, and reasoning capabilities. In our results, Transformer demonstrated stable convergence and the highest success rates, particularly in tasks requiring long-term memory. LSTM also demonstrated competitive performance in specific tasks, showing efficiency despite its simple architecture. In contrast, GPT-2 model appeared to suffer from catastrophic forgetting during fine-tuning, which led to training instability in our experiments. These findings highlight that for specific domains such as text-based games, compact, Transformer architectures can be more effective than fine-tuning general-purpose large-scale language models.

1 Introduction

1.1 Background

Recent advances in Large Language Models (LLMs) have significantly improved the feasibility of controlling computer systems through natural-language instructions. As computational systems become increasingly integrated into daily life and professional workflows, the demand for intuitive, accessible interfaces, where users can issue natural-language instructions, is becoming essential. However, most system operations, such as server administration or software troubleshooting, still require an iterative cycle of issuing specific, formal executable commands, such as shell commands or API calls, and interpreting the resulting terminal output or logs. This semantic gap between natural-language instructions and such formal executable commands often blocks intuitive system operation, as users must master complex syntax and technical feedback to perform even

basic tasks. Bridging this semantic gap is thus critical for enhancing the flexibility and accessibility of modern computing.

To bridge this semantic gap, it is necessary to develop systems capable of automatically converting natural-language instructions into executable commands. Such conversion, however, requires more than mere linguistic translation, as it demands a level of autonomous decision-making that involves observing the environment, maintaining state context, and selecting appropriate action sequences to achieve goals while adhering to common-sense constraints—such as avoiding destructive operations or not giving up prematurely.

Text-based games serve as an ideal testbed for developing an automatic system or an agent to bridge the semantic gap. In these games, both environment observations and agent actions are expressed in natural language, mirroring real-world scenarios where an agent must interpret logs and issue commands. Furthermore, because actions directly influence future states and rewards, these games naturally present sequential decision-making problems that require long-term planning and reasoning.

TextWorld is a text-based game generation framework developed by Microsoft Research that allows for the creation of diverse scenarios with controlled difficulty and task structures [1]. It provides a reproducible benchmark for evaluating autonomous agents in a controlled environment. While many studies have explored language-based agents, there remains a lack of systematic comparisons between different architectures—ranging from traditional recurrent models to pre-trained large-scale models—under identical reinforcement learning (RL) conditions.

1.2 Objectives

In this study, we compare the performance of RL agents. We evaluate three distinct architectures: Long Short-Term Memory (LSTM) [2], Transformer [3], and a pre-trained Generative Pre-trained Transformer 2 (GPT-2) [4]. We implemented a unified RL framework based on Proximal Policy Optimization (PPO) [5] and conducted experiments across three custom scenarios. The objective of this research is to investigate the advantages and limitations of each architecture in bridging the semantic gap between natural-language instructions and executable commands. By training and evaluat-

ing the models under identical conditions, we compare their representational capacity, planning ability, and common-sense reasoning for this conversion task. To evaluate the performance, we use standardized metrics including success rate, cumulative reward, and episode length.

To ensure a fair comparison of the architectures, all action and all objects are listed to unify vocabulary across all models. Furthermore, we masked action vocabulary. Thus, the agent’s action to non- valid commands in the current state is avoided.

2 Method

2.1 Training Architecture

The agent training architecture consists of three main components: the TextWorld environment, the base model with actor-critic heads, and the PPO optimizer, as illustrated in Figure 1.

At each step, the TextWorld environment produces a textual observation. This observation is provided to the base model, either LSTM, Transformer, or GPT-2, which encodes it into a hidden representation. The hidden representation is then forwarded to two separate heads: the actor network, which outputs a probability distribution over actions, and the critic network, which estimates the value of the current state. The actor selects an action from the distribution and executes it in the environment, producing the next observation.

This cycle of interaction between the agent and the environment forms a single “episode,” which serves as the fundamental unit of the reinforcement learning process. An episode terminates when the agent succeeds in the task, fails, or reaches the maximum step limit.

After the interaction phase, the PPO algorithm identifies these collected experiences and computes gradient updates to improve the entire agent, including the base model and both the actor and critic networks, thereby updating the agent’s policy based on the accumulated data.

All three agents are trained using PPO within an Actor-Critic framework [6]. In this framework, the base model, including LSTM, Transformer, or GPT-2, en-

codes the textual observation into a hidden representation vector h , which is then passed to the actor and critic heads.

The actor head is a linear layer that maps h to a logit vector over the action vocabulary. After applying an invalid-action mask to suppress unavailable actions, the logits are converted to a probability distribution via softmax, from which the agent samples an action.

The critic head is a linear layer that maps h to a single scalar value $V(s)$. $V(s)$ represents the estimated cumulative future reward, or value, from the current state.

Table 1 summarizes the architecture of the actor and critic heads for the three models. In all models, each head has one to three linear layers denoted by “Linear($n_{in} \rightarrow n_{out}$)” where n_{in} and n_{out} represent the number of inputs and the number of neuron in the linear layer. “ReLU” represents a rectified linear function. For Transformer model, since the backbone encoder is relatively lightweight with four layers, we employ three linear layers for both heads to provide sufficient expressive power for action selection and value estimation.

For advantage estimation, we employ Generalized Advantage Estimation (GAE) [7]. GAE balances the bias and variance of reward prediction, while PPO stabilizes training by preventing excessive policy updates. The hyperparameters for PPO and GAE shared across all models are shown in Table 2.

2.2 Scenario Design

We developed three distinct custom scenarios, Exploration, Key-Door, and Common-Sense, using the TextWorld framework. Each scenario has a goal that signifies success, and to reach that goal, you must complete multiple sub-goals called quests.

The Exploration scenario focuses on spatial navigation and object discovery. The goal is to navigate a randomly generated map to find and collect a specific target item. The environment’s layout, connectivity, and object placements vary in each episode, requiring the agent to dynamically interpret textual observations.

The Key-Door scenario tests sequential reasoning and planning. The goal is to first find a key, then identify

Table 1: Neural network architecture of the actor and critic heads

Model	Head	Layers	Architecture
LSTM	Actor	1	Linear(256 $\rightarrow$ 106)
	Critic	1	Linear(256 $\rightarrow$ 1)
Transformer	Actor	3	Linear(256 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Linear(128 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Linear(128 $\rightarrow$ 106)
	Critic	3	Linear(256 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Linear(128 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Linear(128 $\rightarrow$ 1)
GPT-2	Actor	1	Linear(768 $\rightarrow$ 106)
	Critic	1	Linear(768 $\rightarrow$ 1)

Table 2: PPO/GAE hyperparameters

Parameter	Value	Description
Discount factor γ	0.99	Discounting future rewards
GAE λ	0.95	Advantage smoothing
Clip ratio ϵ	0.2	Policy update limit
Value loss coef c_v	0.5	State-value objective weight
Batch size	32	Mini-batch size
PPO epochs	4	Iterations per update

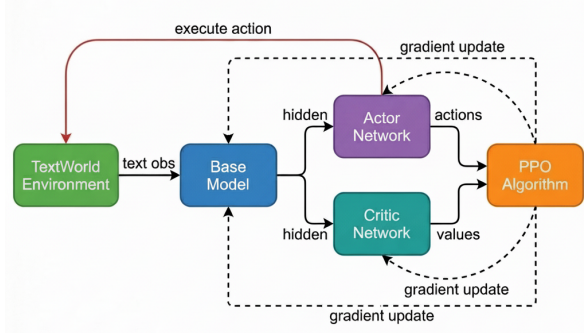


Figure 1: Overview of the TextWorld environment and PPO training loop. The base model with actor-critic heads is called an agent.

and unlock the door, and finally retrieve the “treasure chest” located in the room beyond the door. While the spatial structure is fixed, the agent must learn the precise sequence of actions, such as “take key”, “unlock door with key”, and “open door”, to succeed.

The Common-Sense scenario evaluates the agent’s ability to incorporate common-sense reasoning into its decision-making. The agent starts in a room with various objects, some of which are edible, such as apples and oranges, and others which are not, like poisonous items or inedible tools. The goal is to consume only the edible items. Any attempt to eat an inedible object results in immediate failure reflecting real-world safety constraints.

The scenario parameters are summarized in Table 3. In the table, “Objects” refers to the total number of interactable entities placed within the environment, including target items, keys, doors, and distractors. Here, “Quests” refers to the number of quests. For example, in a Key-Door scenario, it refers to how many doors need to be opened.

Typical room layouts for each scenario are illustrated in Figure 2. In the (a) Exploration scenario, the number of rooms, their connectivity, and object placements are randomly generated for each episode. In contrast, the spatial structures for the (b) Key-Door and (c) Common-

Table 3: Experimental scenario parameters

Scenario	Rooms	Objects	Quests
Exploration	11	3	3
Key-door	5	7	3
Common-sense	2	12	6

Sense scenarios are fixed with object roles remaining invariant across all episodes. This lack of randomness allows agents to potentially learn and memorize specific information placements.

2.3 Base Model Architectures

2.3.1 LSTM

This model includes an embedding layer with a dimension of 128 and a two-layer LSTM with 256 hidden units and 0.2 dropout. The vocabulary size is 10,000 and the maximum input length is 200 tokens. We use a learning rate of 1.0×10^{-3} .

LSTM processes tokenized observations sequentially. The model inputs consist of a fixed-length tensor of tokenized observations and LSTM hidden state for history retention. Observations are preprocessed by removing ASCII art and whitespace, and converted into word-level tokens. The word-level vocabularies were harvested from TextWorld observations and initialized with frequent commands to stabilize early exploration.

2.3.2 Transformer

This model consists of a 4-layer Transformer encoder. We set the dimensionality of the embeddings and the output of all sub-layers to $d_{model} = 256$. The model employs 8 attention heads, a feed-forward network (FFN) dimension of 1024, and a dropout rate of 0.1. Embeddings are scaled by $\sqrt{d_{model}}$ with sinusoidal positional encoding. The vocabulary size is 10,000 and the maximum input length is 512 tokens. The state representation is derived from the last valid position in the sequence using a padding mask. We use a learning rate of 2.0×10^{-4} .

Transformer model utilizes the same preprocessing as LSTM but uses a context-aware history. Inputs include a concatenated history of the previous 2 steps of observations and actions and an attention mask. Contextual markers [HIST], [ACT], and [CUR] distinguish past states, past actions, and the current observation. The word-level vocabularies were harvested from TextWorld observations and initialized with frequent commands to stabilize early exploration.

2.3.3 GPT-2

This model uses a pre-trained gpt2-small base model [4] provided by the Hugging Face Transformers library

[8], which consists of 12 Transformer decoder blocks, a hidden dimension of 768, and 12 attention heads. The total number of parameters is 117 million. For GPT-2, separate learning rates are applied to the Actor/Critic heads and the pre-trained base model. We use 5.0×10^{-4} for the heads and 2.0×10^{-5} for the base model respectively.

GPT-2 employs Byte Pair Encoding (BPE) [9] tokenization and takes the full history-aware token sequence and the history token sequence as inputs. In contrast to other models, GPT-2 uses its default pre-trained BPE tokenizer. Input text is formatted as `Observation: ... \nCommand:`, with a maximum sequence limit of 512 tokens.

2.4 Reward Design

2.4.1 Overall Reward Structure

The reward function is designed to promote exploration while suppressing stagnation and repetitive loops. The reward r_t at step t is calculated as the sum of the base step penalty r_{step} , scenario-specific interim rewards r_{interim} , terminal rewards r_{term} , and auxiliary shaping terms r_{aux} :

$$r_t = r_{\text{step}} + r_{\text{aux}} + r_{\text{interim}} + r_{\text{term}}. \quad (1)$$

The total reward R for an entire episode is defined as the cumulative sum of step-level rewards:

$$R = \sum_{t=1}^T r_t. \quad (2)$$

where T is the total number of steps in the episode.

2.4.2 Step Penalties and Reward Shaping

The base step penalty r_{step} is a constant cost applied at every step to incentivize the agent to reach the goal as

quickly as possible:

$$r_{\text{step}} = -0.01. \quad (3)$$

To prevent stagnation and repetitive loops, r_{aux} includes the following shaping components:

$$r_{\text{aux}} = r_{\text{wait}} + r_{\text{div}} + r_{\text{idle_loop}} + r_{\text{rep}}. \quad (4)$$

The wait penalty r_{wait} is imposed when no environmental progress is detected for more than 10 steps:

$$r_{\text{wait}} = \begin{cases} -0.01, & \text{if no progress for } > 10\text{s,} \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

The diversity bonus r_{div} promotes action variation in the Exploration and Common-sense scenarios:

$$r_{\text{div}} = \begin{cases} +0.01, & \text{if } a_t \neq a_{t-1}, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

where a_t denotes the action taken at step t .

The idle loop penalty $r_{\text{idle_loop}}$ progressively penalizes consecutive idle commands such as `look` or `inventory` in the Key-Door scenario. When the number of consecutive idle commands $n_{\text{idle}} \geq 3$, the penalty is:

$$r_{\text{idle_loop}} = -\min(0.02 \times n_{\text{idle}}, 0.12). \quad (7)$$

Furthermore, a general repetition penalty r_{rep} is applied across all scenarios when any action is selected consecutively. The penalty increases progressively from the third occurrence and is capped at 0.10:

$$r_{\text{rep}} = \min(0.10, (n_{\text{rep}} - 2) \times 0.02), \quad (8)$$

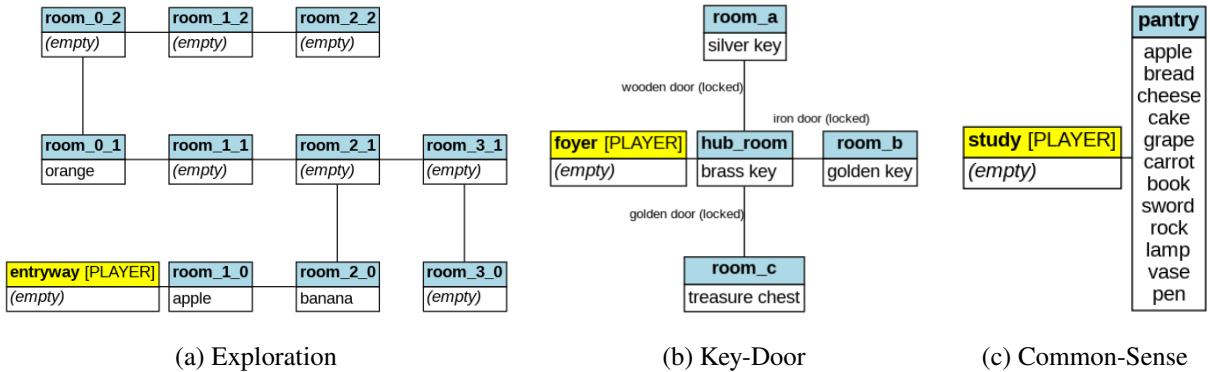


Figure 2: Typical room layouts for each scenario. Each box represents a single room, with the room name at the top and the objects within that room listed at the bottom. Lines between boxes indicate connections between rooms. Where a door exists, its name is noted near the line.

where n_{rep} denotes the number of consecutive executions of the same action.

2.4.3 Scenario-specific Interim Rewards

The scenario-specific interim rewards r_{interim} provide positive feedback for milestone achievements within each scenario. The values for r_{interim} are weighted based on the importance and difficulty of each subgoal in achieving the overall task. By assigning lower rewards to simple actions and higher rewards to complex sequences, we prevent the agent from settling for local optima and motivate it to tackle more challenging objectives necessary for reaching the final goal.

In the Exploration scenario, r_{interim} is +0.15 for discovering a target object and +0.30 for collecting it.

In the Key-Door scenario, r_{interim} is +0.10 for reaching a spoke room, +0.15 for reaching the hub room, and +0.20 for reaching the destination room. Interaction milestones include examining a key of +0.05, discovering a key of +0.20, and taking a key of +0.35. Furthermore, unlocking a door yields +0.35, with an additional +0.15 bonus when using the correct key, opening a door yields +0.25, and passing through the door yields +0.45. Finally, examining the treasure yields +0.05, collecting it yields +0.60, and following each step of the provided walkthrough yields +0.05.

In the Common-sense scenario, r_{interim} is +0.03 for observation of a new object and +0.30 for eating an edible object. Consuming a inedible object incurs an immediate failure penalty of -5.0 , which is equivalent to the terminal failure reward r_{term} and terminates the episode.

2.4.4 Terminal Rewards

The terminal reward r_{term} is granted at the end of an episode, where failure includes instances where the agent interacts with a fatal object or reaches the maximum step limit:

$$r_{\text{term}} = \begin{cases} +10.0, & \text{success,} \\ -5.0, & \text{failure.} \end{cases} \quad (9)$$

3 Experiments

We conducted experiments across all combinations of agents, namely LSTM, Transformer, and GPT-2, and three scenarios: Exploration, Key-Door, and Common-Sense. To ensure reproducibility, each configuration is tested with three independent random seeds 42, 123, and 456, with each agent trained for 500 episodes.

The evaluation is based on three primary metrics: success rate, cumulative reward, and episode length. This evaluation is performed once after the completion of the 500 training episodes to assess the final perfor-

Table 4: Final evaluation metrics.

Model	Scenario	Success Rate (%)	Avg Reward	Avg Ep. Length
LSTM	Exploration	79.3 ± 11.7	8.03 ± 1.89	78.3 ± 14.8
LSTM	Key-Door	100.0 ± 0.0	18.06 ± 0.02	35.0 ± 0.9
LSTM	Common-Sense	99.3 ± 1.2	11.61 ± 0.23	20.0 ± 0.8
Transformer	Exploration	90.0 ± 3.5	9.73 ± 0.55	72.1 ± 8.6
Transformer	Key-Door	90.0 ± 17.3	15.92 ± 3.39	71.2 ± 39.3
Transformer	Common-Sense	97.3 ± 3.1	11.28 ± 0.65	21.3 ± 0.8
GPT-2	Exploration	48.0 ± 16.0	2.82 ± 2.54	116.0 ± 8.1
GPT-2	Key-Door	3.3 ± 4.2	-0.29 ± 0.94	149.1 ± 1.5
GPT-2	Common-Sense	31.0 ± 43.8*	-2.72 ± 9.90	33.1 ± 26.6

*GPT-2 Common-Sense: Only 2 seeds, of 62% and 0%, due to seed 123 crash

mance and convergence. The evaluation phase consists of 10 episodes conducted with the three scenarios. For policy evaluation, GPT-2 employs deterministic action selection, whereas LSTM and Transformer utilize stochastic selection. As defined in Section 2.4, the maximum step limit for an episode is 150; reaching this limit without satisfying the objective is treated as a failure.

4 Results and Analysis

4.1 Overall Performance

Of the 27 training runs, comprising 3 models × 3 scenarios × 3 seeds, 26 completed successfully and one, GPT-2 Common-Sense run with seed 123, crashed. All trainings were run for the full 500 episodes.

Figure 3 shows the success rate as a function of the number of training episodes for the three scenarios. The blue, orange, and green lines show the result for LSTM, Transformer, and GPT-2, respectively. The colored shaded region for each agent represents the standard deviation across 3 seeds. Figure 4 shows the average episode length as a function of the number of training episodes for the three scenarios. The blue, orange, and green lines show the result for LSTM, Transformer, and GPT-2, respectively. The colored shaded region for each agent represents the standard deviation across 3 seeds.

The final evaluation metrics are shown in Table 4. In this table, the second, the third, and the fourth columns show the average of the percentage archiving the goal, the average of the cumulative rewards per episode, and the average number of steps taken per episode. These results also show the standard deviation across three random seeds.

LSTM achieved the highest success rates on Key-Door and Common-Sense, while Transformer demonstrated stable and high performance across all scenarios, including Exploration. In contrast, GPT-2 performed poorly across all scenarios.

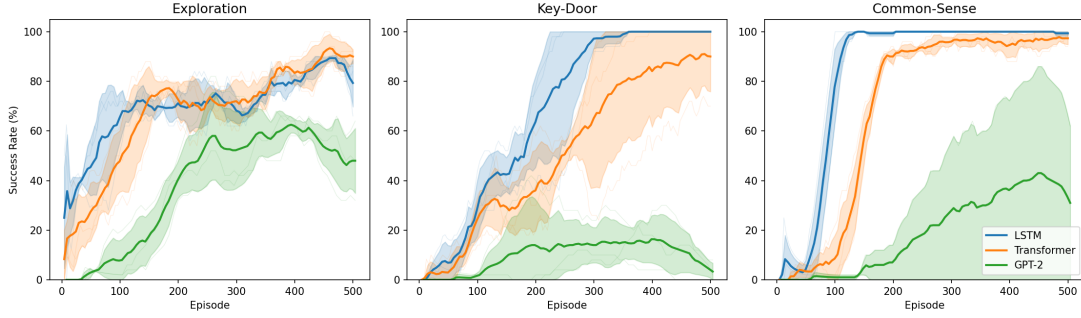


Figure 3: Training success rate over episodes.

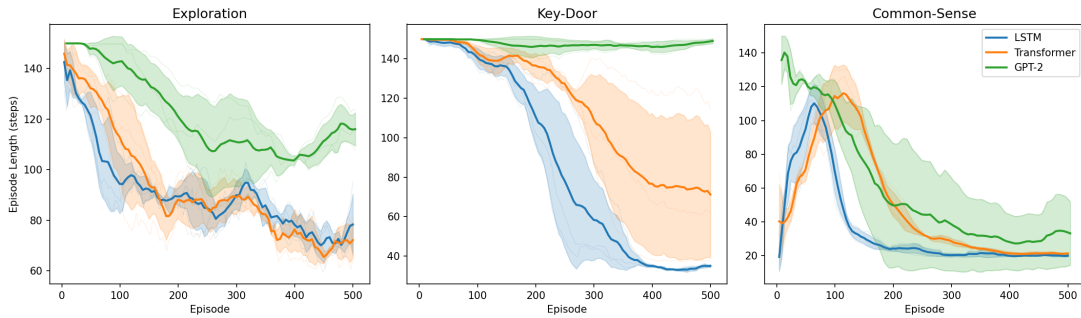


Figure 4: Training episode length over episodes.

4.2 Per-Scenario Analysis

4.2.1 Exploration

Transformer achieved the highest average success rate of 90.0%. LSTM reached 79.3% success rate on this scenario.

GPT-2 performed at 48.0%, lower than LSTM, indicating that 124 M parameters of pre-trained knowledge provided no advantage in spatial navigation over LSTM trained from scratch.

4.2.2 Key-Door

Transformer achieved 90.0% average success rate on this scenario. At seed 42 and seed 456, it reached 100% success rate with mean reward 18.06 and mean episode length 37.4 steps. Seed 123 achieved only 70%, showing inter-seed variance with a standard deviation of 17.3%.

LSTM achieved 100.0% success rate on Key-Door across all three seeds, achieving a reward of 18.06, closely approximating the 16-step walkthrough.

GPT-2 achieved only 3.3% on Key-Door, a catastrophic failure. The precise 16-step walkthrough proved impossible for GPT-2.

4.2.3 Common-Sense

Transformer achieved 97.3% average success rate with a mean episode length of 21.3 steps, which is about

2.4× the length of the 9-step walkthrough, indicating near-optimal behavior.

LSTM achieved 99.3% success rate, approaching Transformer performance. LSTM successfully learned to discriminate edible from inedible objects using reward signals alone.

GPT-2 achieved only 31.0% on average. Seed42 reached 62% during training, seed456 ended at 0%, and seed123 crashed. This represents catastrophic forgetting known as catastrophic forgetting during PPO fine-tuning.

4.3 Training Dynamics

In Key-Door scenario with the seed 42, the loss value of Transformer agent at the episode 484 is 0.13. At the end of the training, the loss value is 0.16. This means Transformer agent demonstrated stable convergence behavior at the final stage of the training.

In contrast, GPT-2 agent demonstrated unstable training behavior in Common-Sense scenario. With the seed 42, we found that the loss values at the episodes 472, 496, 504 were 3.81, 15.9, and 18.8, respectively. The loss exploded roughly 5-fold in this training, and the success rate that had reached 80% at the episode 472 but it regressed to 62% at the end of the training.

5 Discussion

5.1 Trade-offs Between Generalization and Memorization

The result that LSTM of 92.9% slightly outperformed Transformer of 92.4% in average success rate might seem to contradict the theoretical superiority of attention mechanisms. However, a detailed analysis of the scenarios' characteristics, specifically the presence or absence of randomness, explains this outcome as a contrast between an agent's generalization capability and a recurrent network's memorization capacity.

First, we discuss deterministic structures and LSTM memorization. The Key-Door and Common-Sense scenarios feature fixed room layouts and consistent object roles in a linear structure. In such static environments, a small-scale recurrent model like LSTM can effectively perform rote learning of the optimal action sequences. The 100% success rate and 0.0 standard deviation in the Key-Door scenario demonstrate that LSTM perfectly memorized the deterministic task purely from reward signals.

Second, we consider Transformer superiority in random map environments. In contrast, the Exploration scenario randomly generates room layouts and object placements. In this dynamic environment, memorizing specific action sequences is ineffective, necessitating a generalization capability to dynamically understand spatial structures from the observation. Here, Transformer with 90.0% success, capable of relational understanding between tokens through self-attention, significantly outperformed LSTM with 79.3%. This suggests that Transformer learned a general exploration strategy independent of specific information placement.

Finally, we address structural expressiveness and overfitting risks. While Transformer's 3-layer MLP heads and self-attention provide high representational power, its inability to match LSTM in Key-Door, indicated by a large standard deviation of 17.3%, is noteworthy. This suggests that the high expressiveness may have led to overfitting to non-essential descriptive details within specific training episodes, causing policy instability compared to the simpler LSTM.

5.2 GPT-2's Unexpectedly Poor Performance

One of the most significant findings of this study is that the 124 M-parameter pre-trained GPT-2 achieved the lowest performance.

First, Common-Sense was designed as the scenario where GPT-2's pre-trained knowledge, for instance "apples are food", would be most beneficial. Yet GPT-2 scored 0% on this scenario with the seed 456. The

temporary rise to 80% during training followed by collapse suggests that pre-trained knowledge was initially utilized but destroyed during PPO fine-tuning, known as catastrophic forgetting.

Second, we examine evidence of catastrophic forgetting. The abrupt loss increase from 3.81 to 18.75 strongly suggests that PPO policy updates destroyed the backbone's pre-trained representations.

Finally, we discuss the challenge of randomly initialized heads. The Actor/Critic heads are randomly initialized, meaning that a 768 to 106 linear transformation performs near-random action selection initially. For tasks like Key-Door that require long, precise action sequences, 500 episodes may be insufficient for adequate head training.

6 Conclusion and Future Work

6.1 Conclusion

In this study, we compared the performance of three distinct architectures-LSTM, Transformer, and GPT-2-as PPO-based RL agents to evaluate their effectiveness in bridging the semantic gap between natural language instructions and executable commands. To this end, we designed three scenarios, Exploration, Key-Door, and Common-Sense, on the TextWorld environment. Under unified PPO stabilization mechanisms and evaluation protocol, we obtained several key findings.

First, regarding the high performance of simple architectures, LSTM and Transformer demonstrated equally high overall performance, proving their excellent suitability for text-based cognitive tasks. Transformer showed consistent stability, with a success rate exceeding 90% across all scenarios, while LSTM achieved peak performance, reaching up to 100%, on Key-Door and Common-Sense. This indicates that architectures designed for sequence modeling are highly effective for these tasks.

Second, we observed limitations of large-scale pre-trained models. The pre-trained GPT-2 Small achieved the lowest performance among the three models. Its 124 M parameters could not be effectively leveraged within 500 episodes of PPO fine-tuning, and the model suffered from catastrophic forgetting in the Common-Sense scenario. This result highlights that an increased parameter count does not directly translate to improved performance in specialized RL environments.

Finally, we discuss architectural alignment with task complexity. The fact that small-to-mid-size models significantly outperformed the massive GPT-2 suggests an imbalance between model scale and task complexity. For constrained cognitive tasks in text-based reinforcement learning, selecting a model capacity that aligns

with the task's complexity is more critical than simply utilizing massive pre-existing weights.

6.2 Future Work

To build upon the findings of this study, several directions for future research are identified.

First, a priority is refining fine-tuning for large models. Improving the training of models like GPT-2 is a priority. Future work should explore more conservative fine-tuning strategies to preserve pre-trained knowledge while learning new tasks.

Second, we consider scaling and multi-task learning. Scaling up the training process could reveal the threshold where pre-trained knowledge becomes beneficial. Additionally, training a single agent on multiple scenarios simultaneously could promote cross-scenario transfer and improve overall cognitive flexibility.

Finally, we plan integration with advanced models and structures. Comparing current results with more recent open-source LLMs and exploring graph-based approaches could strengthen structural reasoning capabilities. Combining graph-integrated architectures with the representational power of Transformers remains a promising avenue for enhancing text-based agents.

References

- [1] M.A. Côté, Ákos Kádár, X. Yuan, B. Kybartas, T. Barnes, E. Fine, J. Moore, R.Y. Tao, M. Hausknecht, L.E. Asri, M. Adada, W. Tay, and A. Trischler, "Textworld: A learning environment for text-based games," arXiv preprint arXiv:1806.11532, 2019.
- [2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol.9, no.8, pp.1735–1780, 1997.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv preprint arXiv:1706.03762, 2023.
- [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, "Language models are unsupervised multitask learners," OpenAI blog, 2019.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.
- [6] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," *Proceedings of The 33rd International Conference on Machine Learning*, ed. M.F. Balcan and K.Q. Weinberger, *Proceedings of Machine Learning Research*, vol.48, New York, New York, USA, pp.1928–1937, PMLR, 20–22 Jun 2016.
- [7] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," arXiv preprint arXiv:1506.02438, 2018.
- [8] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T.L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A.M. Rush, "Huggingface's transformers: State-of-the-art natural language processing," arXiv preprint arXiv:1910.03771, 2020.
- [9] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ed. K. Erk and N.A. Smith, Berlin, Germany, pp.1715–1725, Association for Computational Linguistics, aug 2016.